Ex:No: 5A Date: 01/04/26	Smart Parking Slot Occupancy Classification Using K-Nearest Neighbor Algorithm

Problem Description:

In a smart city environment, parking management systems collect sensor data to determine whether parking slots are **Occupied** or **Vacant**. Each parking record includes numerical attributes such as **Distance from entry gate, Time of day, Vehicle density,** and **Parking duration.**

The objective is to classify parking slot status using the **K-Nearest Neighbor (KNN)** algorithm and visualize the classification results for efficient parking management.

Objective:

To implement and visualize the K-Nearest Neighbor algorithm in R for classifying parking slot occupancy based on sensor data.

Procedure:

- **Step 1: Create Parking Dataset**

	Distance	TimeOfDay	VehicleDensity	ParkingDuration	Status
1	50	9	20	30	Occupied
2	120	14	60	90	Occupied
3	80	11	45	60	Occupied
4	30	8	15	20	Vacant
5	200	18	80	120	Occupied
6	60	10	30	45	Vacant
7	150	16	70	100	Occupied
8	40	7	10	15	Vacant
9	100	13	55	70	Occupied
10	170	19	75	110	Occupied

```
parking_data <- data.frame(
Distance = c(50, 120, 80, 30, 200, 60, 150, 40, 100, 170),
TimeofDay = c(9, 14, 11, 8, 18, 10, 16, 7, 13, 19)
VehicleDensity = c(20, 60, 45, 15, 80, 30, 70, 10, 55, 75),
ParkingDuration = c(30, 90, 60, 20, 120, ~~120~~ 45, 100, 15, 70, 110),
Status = c("Occupied", "Occupied", "Occupied", "Vacant",
           "Occupied", "Vacant", "Occupied", "Vacant",
           "Occupied", "Occupied")
)
parking_data
```

OUTPUT:

	Distance	Time of Day	Vehicle Density	Parking Duration	Status
1	50	9	20	30	Occupied
2	120	14	60	90	Occupied
3	80	11	45	60	Occupied
4	30	8	15	20	Vacant
5	200	18	80	120	Occupied
6	60	10	30	45	Vacant
7	150	16	70	100	Occupied
8	40	7	10	15	Vacant
9	100	13	55	70	Occupied
10	170	19	75	110	Occupied

- **Step 2: Select Features and Class Label**

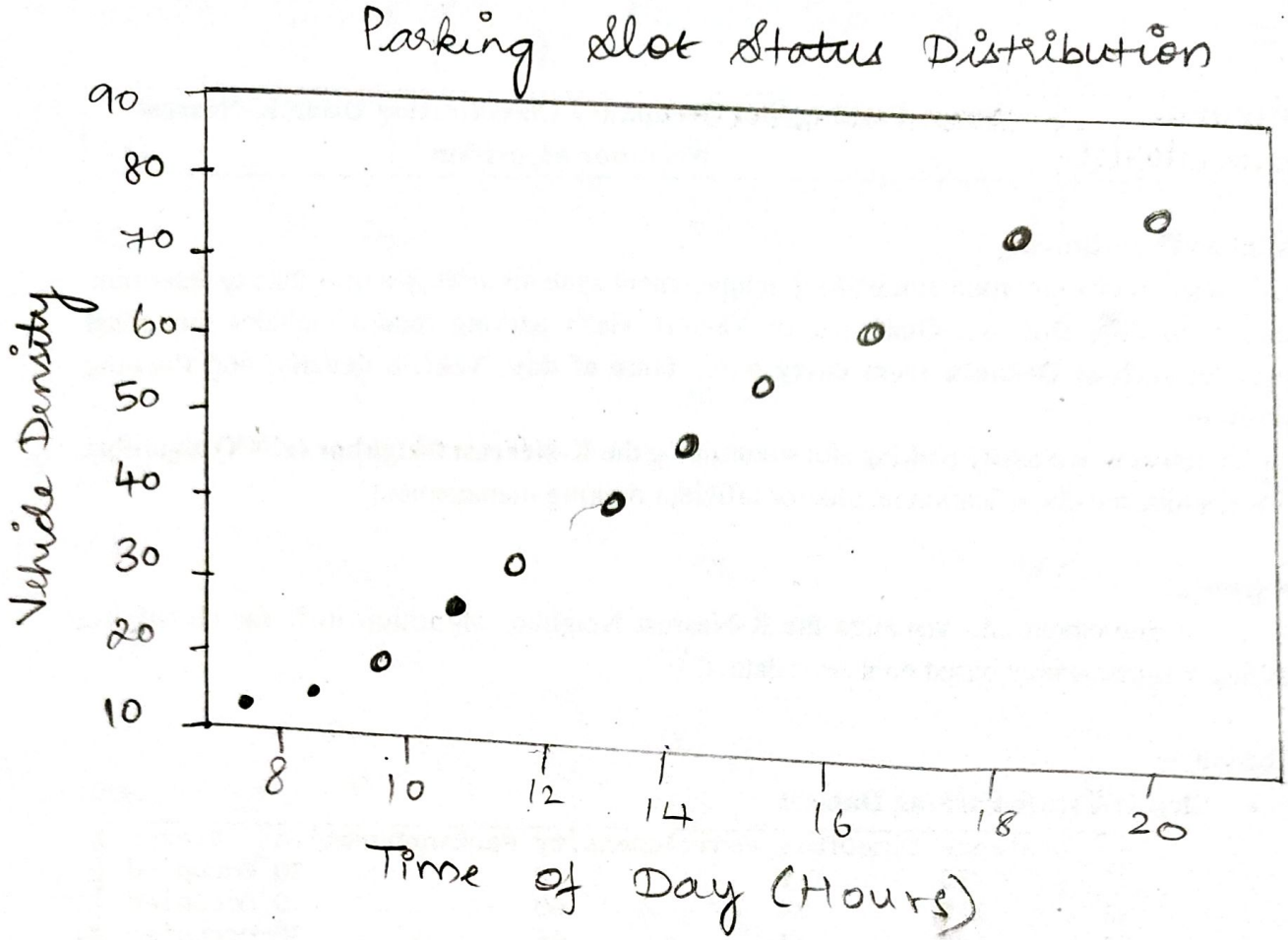
```
features <- parking_data [, 1:4]
labels <- as.factor (parking_data $ status)
```

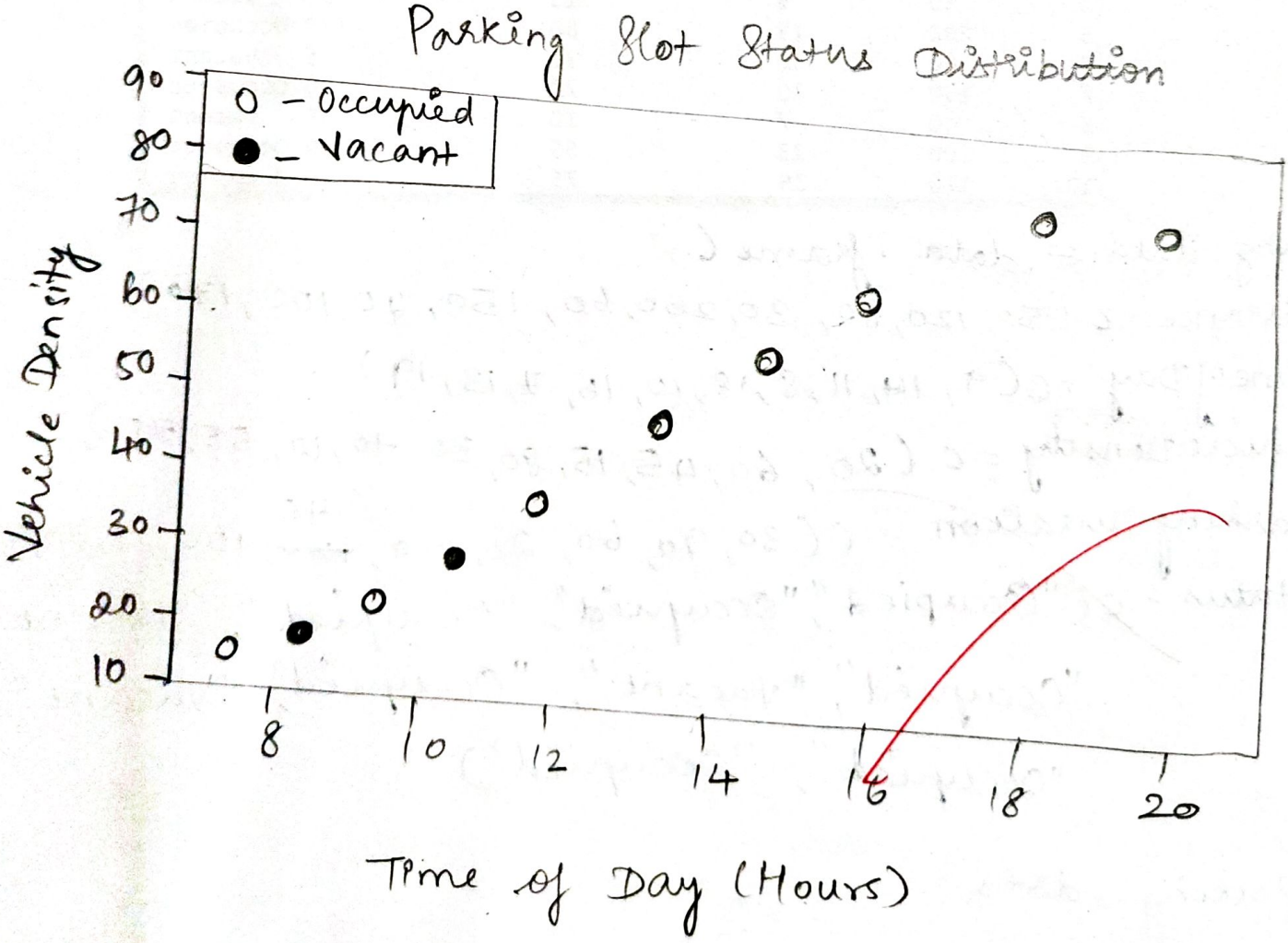
- **Step 3: Data Visualization (Scatter Plot) – TimeOfDay and VehicleDensity,**

```
plot (features $ Time of Day , features $ vehicleDensity,
      col = as. numeric (labels),
      pch = 19,
      xlab = " Time of Day (Hours)",
      ylab = "Vehicle Density",
      main = " Parking Slot status Distribution")
legend ("topleft",
        legend = levels(labels),
        col = 1:2
        pch = 19)
```

- **Step 4: Normalize the Data**

```
normalise <- function (x) {
  (x-min(x)) / (max (x) - min (x))
}
features . norm <- as. data. frame (lapply (features,
                                        normalize))
```


Parking Slot Status Distribution
Vehicle Density
90
80
70
60
50
40
30
20
10
8
10
12
14
16
18
20
Time of Day (Hours)


Parking Slot Status Distribution
O - Occupied
● - Vacant
Vehicle Density
90
80
70
60
50
40
30
20
10
8
10
12
14
16
18
20
Time of Day (Hours)

- **Step 5: Train(70%) – Test Split(30%)**

```
index <- sample(1:nrow(features_norm), 0.7*
                nrow(features_norm))
train_data <- features_norm[index, ]
test_data <- features_norm[-index, ]
train_labels <- labels[index]
test_labels <- labels[-index]
```

- **Step 6: Apply KNN Algorithm – k=3 and predict labels**

```
library(class)
K <- 3
Predicted_labels <- knn(
  train = train_data,
  test = test_data,
  cl = train_labels,
  k = K
)
```

- **Step 7: Confusion Matrix and Accuracy**

```
confusion_matrix <- table(
  Actual = test_labels,
  predicted = predicted_labels
)
confusion_matrix
accuracy <- sum(diag(confusion_matrix)/
                sum(confusion_matrix)
accuracy.
```

> predicted_labels

[1] Occupied Vacant Occupied
Levels: Occupied Vacant

> confusion_matrix

	Predicted	
Actual	Occupied	Vacant
Occupied	2	0
Vacant	0	1

> accuracy

[1] 1

- **Step 8: Visualization of Classification Result – Normalized data - TimeOfDay and VehicleDensity,**

```
Plot (test_data$ TimeOfDay, test_data$ VehicleDensity,
col =as.numeric (predicted_labels),
pch = 19,
Xlab = "Time of Day (Normalized)",
ylab= "Vehicle Density (Normalized)",
main = "KNN Classification Result for Parking Slots")
legend ("topleft",
        legend (levels (predicted_labels),
        col = 1:2,
        Pch =19)
```

- **Step 8: Pairwise Visualization using ggpairs:** To visualize relationships between multiple numerical attributes and observe how they vary with parking slot status.

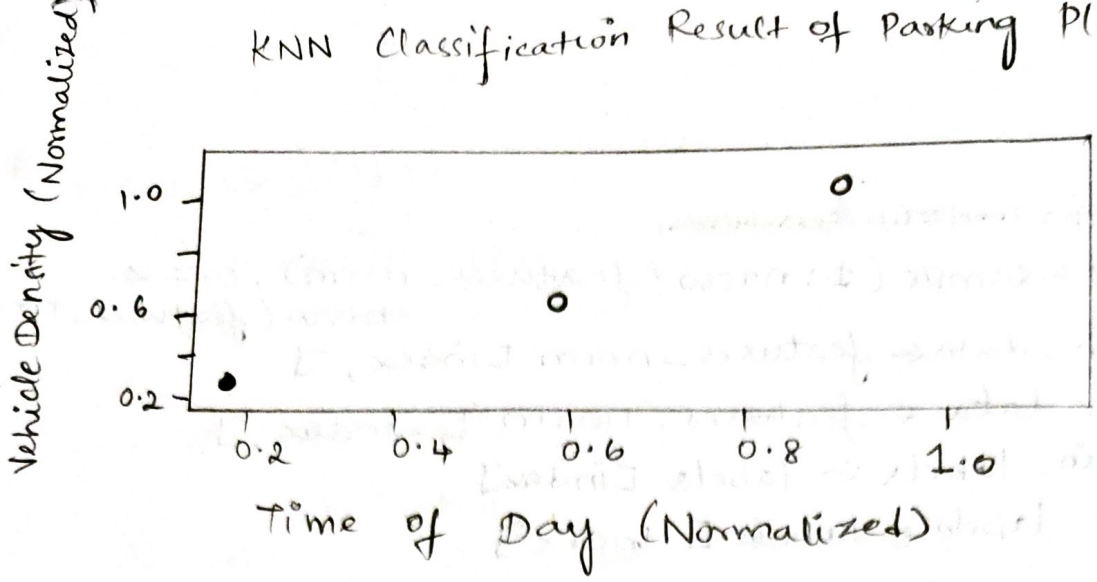
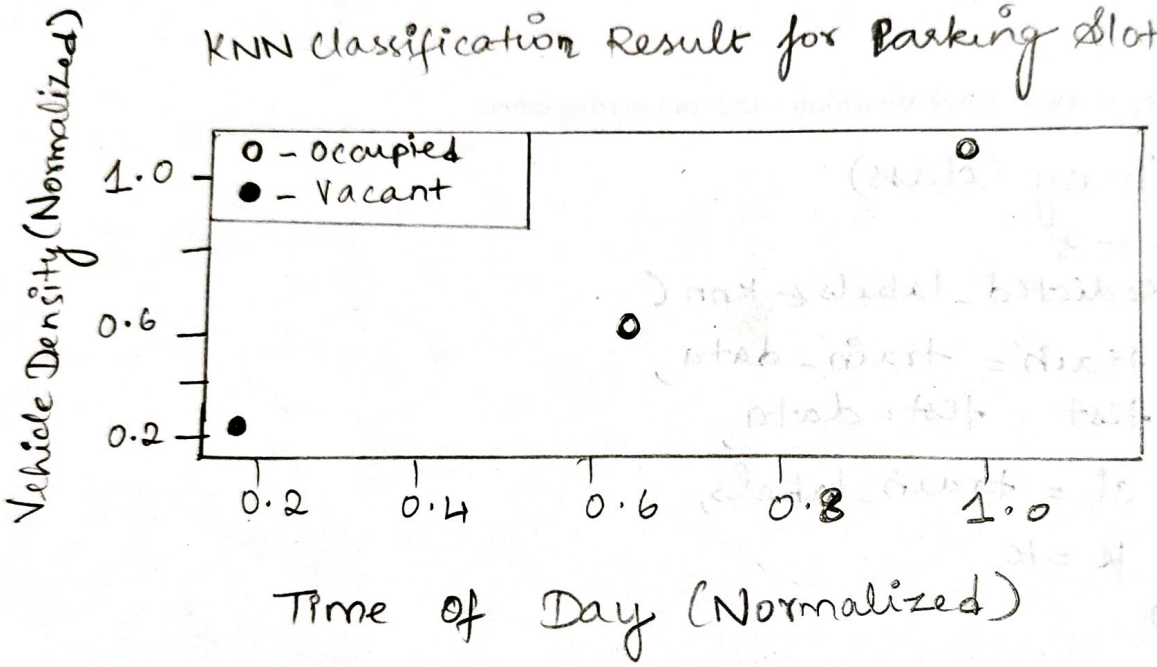
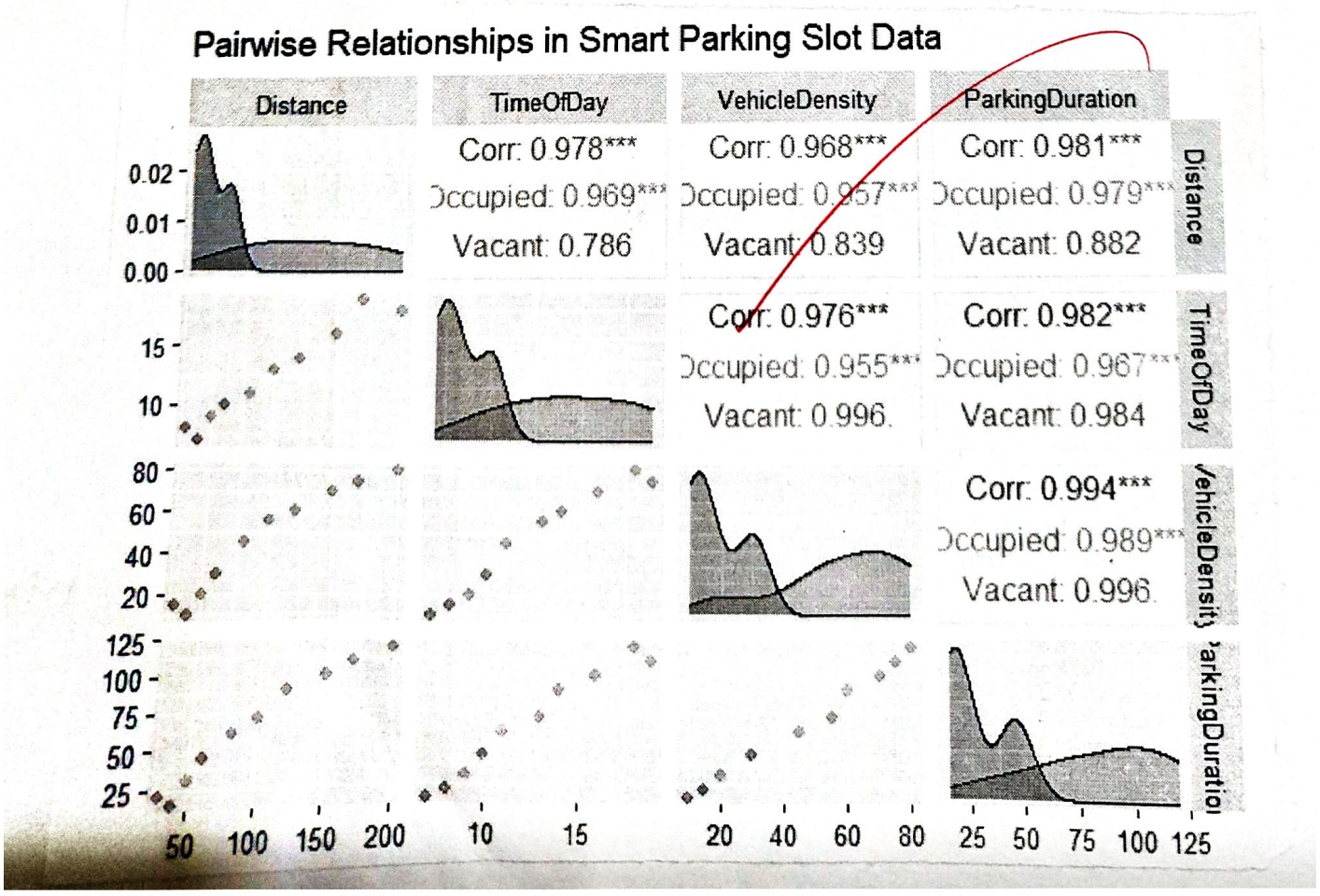
```
install. packages ("GGally")
library (GGally)
ggpairs (
  parking_data,
  columns=1:4,
  aes (color= status, alpha = 0.7),
  title = " Pairwise Relationships in smart
            parking slot Data"

)
```

Result:

Thus, the program successfully demonstrates the use of the K-Nearest Neighbor algorithm in R to classify smart parking slot occupancy. The visualizations effectively represent parking patterns, and the model provides accurate classification suitable for smart city traffic and parking management systems.

KNN Classification Result of Parking Plots



KNN Classification Result for Parking Slots